

SIMATS ENGINEERING

ASSESSMENT TOOL 2

Case study

COURSE NAME: Computer Networks

COURSE CODE: CSA0711

COURSE OUTCOME COVERED

CO3: Analyse the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions

Assessment Tool Weightage: 40%

Code of Conduct:

I **K. Sampath Reddy (192424285)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **K. Sampath Reddy**

QUESTIONS: 4

Total Marks: 40

Case Study 1

A multinational IT company uses a cloud-based video conferencing platform for daily client meetings. During peak business hours, employees experience frequent voice distortion, frozen video frames, and delayed communication. Network monitoring reports indicate an 8% packet loss, 180 ms jitter, and fluctuating bandwidth utilization. The IT administrator suspects that the issue is related to the transport protocol or application-level buffering strategy.

Questions

- Analyze whether the problem is primarily caused by the transport protocol (TCP/UDP) or the application layer.
- Justify your answer based on the communication requirements of real-time multimedia applications.
- Recommend suitable protocol-level or application-level improvements to enhance conferencing quality.

Case Study 2

A multinational software company hosts its web services across multiple continents. Employees frequently report long delays before the application homepage loads. Network analysis shows that the average DNS lookup time is approximately 800 ms, significantly affecting user experience. The organization plans to redesign its DNS infrastructure while ensuring continuous service availability during server failures.

Questions

- Analyze the reasons behind the high DNS resolution time.
- Propose a suitable DNS architecture using techniques such as Anycast DNS, DNS pre-fetching, and TTL optimization to reduce the lookup time below 50 ms.

- Evaluate the advantages and limitations of your proposed architecture in terms of performance, scalability, and availability.

Case Study 3

A cloud service provider delivers applications to millions of users worldwide. During promotional events, users experience intermittent delays in accessing cloud-hosted services. Investigation reveals that DNS queries are taking nearly 800 ms because all requests are directed to a single primary DNS server. The company wants a highly available DNS solution capable of handling global traffic efficiently.

Questions

- Examine the impact of centralized DNS architecture on application performance.
- Design a distributed DNS solution that minimizes lookup latency while maintaining high availability.
- Analyze how TTL values, Anycast routing, and DNS caching influence system performance and fault tolerance.

Case Study 4

A financial institution operates an electronic stock trading platform where thousands of buy and sell orders are submitted every second. During market opening hours, the average order acknowledgment latency increases from 1 ms to 40 ms, resulting in delayed trade confirmations and customer dissatisfaction. Network engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

Questions

- Analyze three possible TCP-related factors responsible for the latency increase, considering flow control, Nagle's algorithm, and SYN queue limitations.
- Explain how each factor affects transaction processing in high-frequency systems.

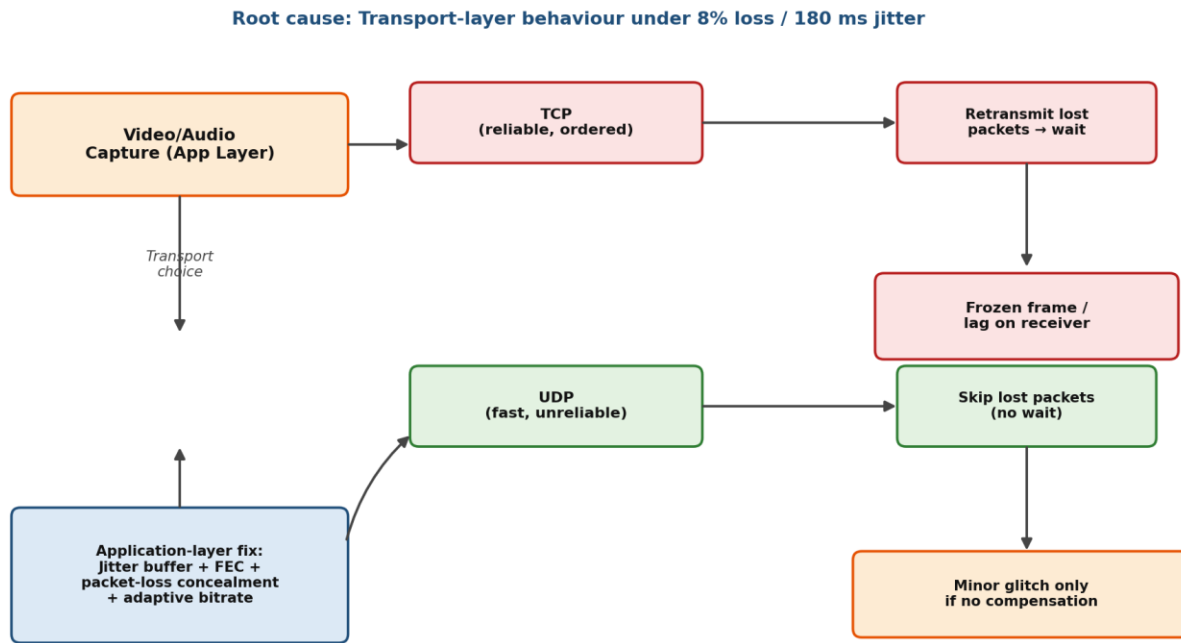
- Recommend appropriate TCP configuration changes to reduce latency and improve transaction reliability.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

ANSWERS

Case Study 1: Video Conferencing Quality Issues



1. Transport Layer vs Application Layer

The problem is primarily a transport-layer issue rather than an application-layer one, though the two are closely linked. The symptoms described — voice distortion, frozen frames, delayed communication, 8% packet loss, and 180 ms jitter — point directly to how packets are being delivered across the network, which is the core responsibility of the transport protocol in use.

- If the platform uses TCP, its reliability mechanisms (retransmission, in-order delivery, congestion control, slow-start after loss) force the receiver to wait for lost segments to be resent before handing data to the application. At 8% loss this

happens constantly, converting packet loss directly into added delay — explaining the frozen frames and lag employees observe.

- If UDP is used without adequate application-layer compensation (jitter buffers, forward error correction, adaptive bitrate), the 8% loss and 180 ms jitter will surface directly as glitches, distortion, and out-of-order audio, since UDP offers no delivery or ordering guarantees of its own.
- Fluctuating bandwidth utilization is a strong indicator of network congestion causing the loss in the first place — a transport/network-level symptom, not an application logic defect. However, how gracefully the application buffers and conceals that loss (jitter buffer sizing, codec resilience) is what ultimately determines whether the user experiences a stutter or a freeze — so the application-level buffering strategy is a secondary, amplifying factor.
- Conclusion: the underlying cause is transport-layer behaviour interacting badly with network conditions; the application-layer buffering strategy determines how severely that root cause is felt by the user.

2. Justification Based on Real-Time Multimedia Requirements

- Real-time audio/video is delay-sensitive but loss-tolerant: a slightly distorted or missing frame is acceptable and often imperceptible, but a late frame breaks the real-time experience entirely (it arrives after its playback deadline and is useless).
- TCP guarantees delivery and order but not timeliness — its retransmission and congestion-control back-off (halving the send rate on loss) directly conflict with real-time needs, causing head-of-line blocking and stalls (frozen video) whenever loss occurs, exactly as reported.
- UDP is timely but unreliable — it matches real-time needs only when paired with application-level correction (jitter buffering, packet-loss concealment, FEC), which is why the buffering strategy at the application layer also matters in tandem with the transport choice.

- Quantitatively, ITU-T recommends one-way latency under ~150 ms for good conferencing quality; TCP retransmission round trips under 8% loss can easily exceed this, while UDP with a well-tuned ~180 ms jitter buffer can still deliver acceptable, if imperfect, real-time audio/video.
- Hence the root cause is the mismatch between the chosen transport behaviour and the low-latency, loss-tolerant nature of multimedia traffic — real-time media should never be forced to wait for a retransmission.

3. Recommended Improvements

- Protocol level: Use UDP-based real-time transport (e.g., RTP/SRTP) instead of TCP for audio/video streams so late packets are simply skipped rather than causing stalls.
- Use congestion-aware real-time protocols (e.g., WebRTC's RTP over SCTP/DTLS with adaptive bitrate and RTCP feedback) so the sender reacts to the fluctuating bandwidth reported.
- Application level: Implement adaptive jitter buffers sized dynamically based on measured jitter (180 ms) to smooth playout without adding excessive delay.
- Apply Forward Error Correction (FEC) and packet-loss concealment algorithms to mask the 8% loss without needing retransmission.
- Use adaptive bitrate/resolution scaling (simulcast/SVC) so the encoder reduces quality gracefully under congestion instead of causing freezes.
- Prioritize real-time traffic using QoS/DSCP marking and traffic shaping on the network to reduce jitter and loss during peak business hours.
- Keep TCP only for signaling/control traffic (call setup, chat) where reliability matters more than timeliness, reinforcing the correct division of transport responsibilities.

Case Study 2: High DNS Resolution Time

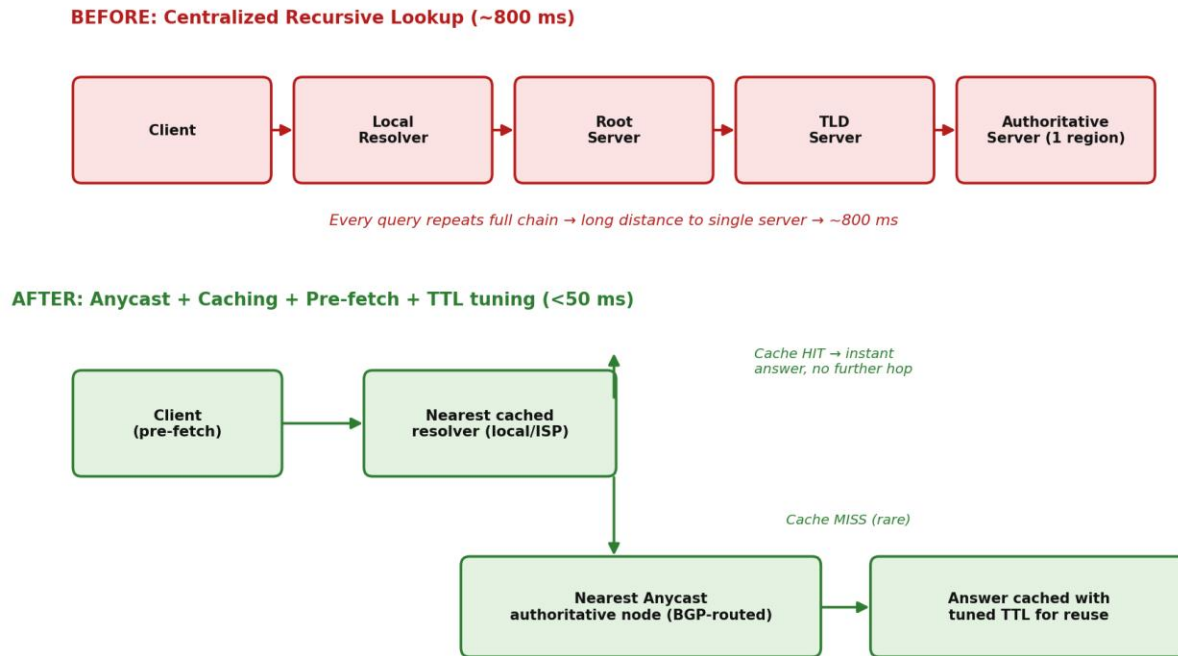


Fig 2.1 — Full recursive chain today (~800 ms) versus Anycast + caching + pre-fetch design (<50 ms)

1. Reasons for High DNS Resolution Time

- Geographically distant DNS servers: if all queries are resolved by a small number of centrally located name servers, users on other continents incur long round-trip propagation delay simply due to physical distance.
- Lack of caching or a low cache hit rate: if TTL values are set poorly or resolvers are not caching effectively, every request needs a fresh recursive lookup through the full DNS hierarchy (root → TLD → authoritative), each hop adding tens to hundreds of milliseconds.
- No use of Anycast: a single unicast IP for authoritative servers means every user worldwide is routed to the same physical server rather than the nearest one.

- Recursive resolution overhead: multiple sequential round trips between the local resolver, root servers, TLD servers, and authoritative servers add cumulative latency, especially without pre-fetching or a warm cache.
- Server load and no redundancy: a single point of resolution can become a queuing bottleneck during high traffic, further inflating the measured ~800 ms.
- No use of pre-fetching: browsers/apps that resolve domains only at the moment of need (rather than in advance) expose the full lookup latency directly to the user.

2. Proposed DNS Architecture

A redesigned architecture combining Anycast DNS, DNS pre-fetching, and TTL optimization can reduce lookup time below 50 ms:

- Anycast DNS: Deploy authoritative name servers at multiple geographically distributed points of presence (PoPs) advertising the same Anycast IP, so each user's query is automatically routed to the nearest server via BGP, cutting propagation delay.
- DNS pre-fetching: Have client browsers/applications resolve likely-needed domains in advance (e.g., while the page is loading), so the DNS answer is already cached locally when the actual request is made.
- TTL optimization: Set moderate TTL values (e.g., 300–3600 seconds depending on how often records change) so that resolvers cache answers long enough to avoid repeated full resolution, while still allowing timely updates during failover.
- Deploy multiple redundant authoritative servers behind the same Anycast address for automatic failover if one location goes down, ensuring continuous availability.
- Use recursive resolvers with local caching layers close to users (e.g., ISP-level or CDN-integrated resolvers) to shorten the resolution path further.

3. Advantages and Limitations

- Performance: Anycast plus caching and pre-fetching can bring typical lookups from ~800 ms down to tens of milliseconds by removing distance and repeated-lookup overhead.
- Scalability: Anycast naturally load-balances across nodes as traffic grows, and additional PoPs can be added without changing the DNS IP.
- Availability: Because Anycast automatically reroutes to the next-nearest healthy node if one fails, the design is resilient to single-server or regional outages.
- Limitation — TTL trade-off: shorter TTLs improve failover speed but increase query volume/load; longer TTLs reduce load but slow propagation of changes (e.g., during migrations).
- Limitation — Anycast complexity: requires BGP routing control and multiple PoPs, adding operational and infrastructure cost.
- Limitation — stale caches: aggressive pre-fetching or caching can occasionally serve slightly outdated records if TTLs are set too high.

Case Study 3: Centralized DNS and Global Latency

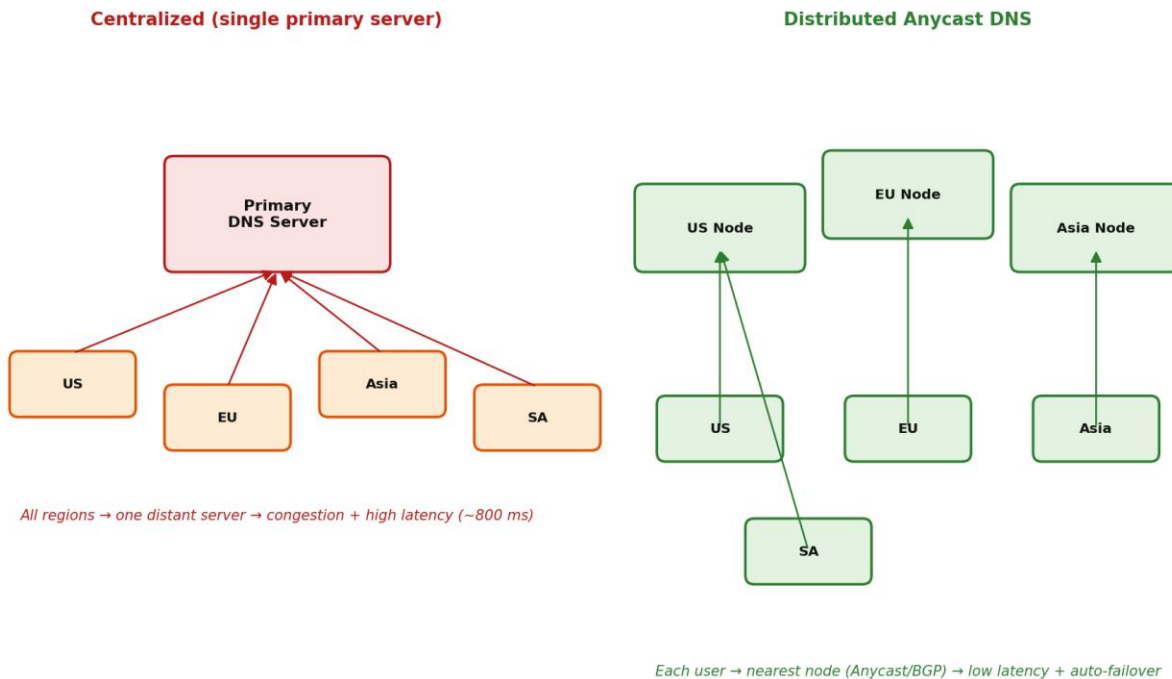


Fig 3.1 — Centralized single-server DNS (bottleneck) versus distributed Anycast DNS (each user served by nearest node)

4. Impact of Centralized DNS Architecture

- All queries converge on one primary server, so response time grows with distance for users far from that server, explaining the ~800 ms lookup time.
- The single server becomes a performance bottleneck during traffic spikes (e.g., promotional events), causing queuing delay and intermittent timeouts.
- It is a single point of failure: if the primary server is overloaded or goes down, DNS resolution — and therefore access to all dependent services — fails globally.
- There is no natural load distribution, so traffic surges disproportionately degrade performance for all users simultaneously rather than being absorbed regionally.

5. Distributed DNS Solution Design

- Deploy authoritative DNS servers at multiple geographically distributed locations, all sharing one Anycast IP address so queries are routed to the nearest healthy server via BGP.
- Add secondary/replica name servers in each region with automatic zone-transfer synchronization to keep records consistent across all nodes.
- Introduce hierarchical caching resolvers (ISP-level and CDN edge resolvers) so most repeat queries are answered locally without reaching the authoritative servers at all.
- Implement health checks and automatic failover so that if one node becomes unavailable, Anycast routing withdraws its advertisement and traffic shifts seamlessly to the next-nearest node.
- Apply load-balancing/traffic-shaping at each PoP to absorb promotional-event traffic spikes locally instead of funneling them to one server.

6. Influence of TTL, Anycast, and Caching

- TTL values: Lower TTLs allow faster propagation of changes and quicker failover but increase the number of queries hitting authoritative servers; higher TTLs reduce load but slow down updates during outages or migrations — a balance is needed (e.g., short TTLs for critical/failover records, longer TTLs for stable records).
- Anycast routing: Distributes load across nodes based on network proximity, reduces latency by serving users from the nearest PoP, and improves fault tolerance since traffic automatically reroutes around failed nodes.
- DNS caching: Reduces repeated round trips to authoritative servers, lowers overall latency and server load, and helps sustain performance during traffic surges — though it must be tuned with appropriate TTLs to avoid serving stale data.

- Together, these three mechanisms transform a fragile single-server design into a low-latency, self-healing, horizontally scalable DNS system.

Case Study 4: TCP Latency in High-Frequency Trading

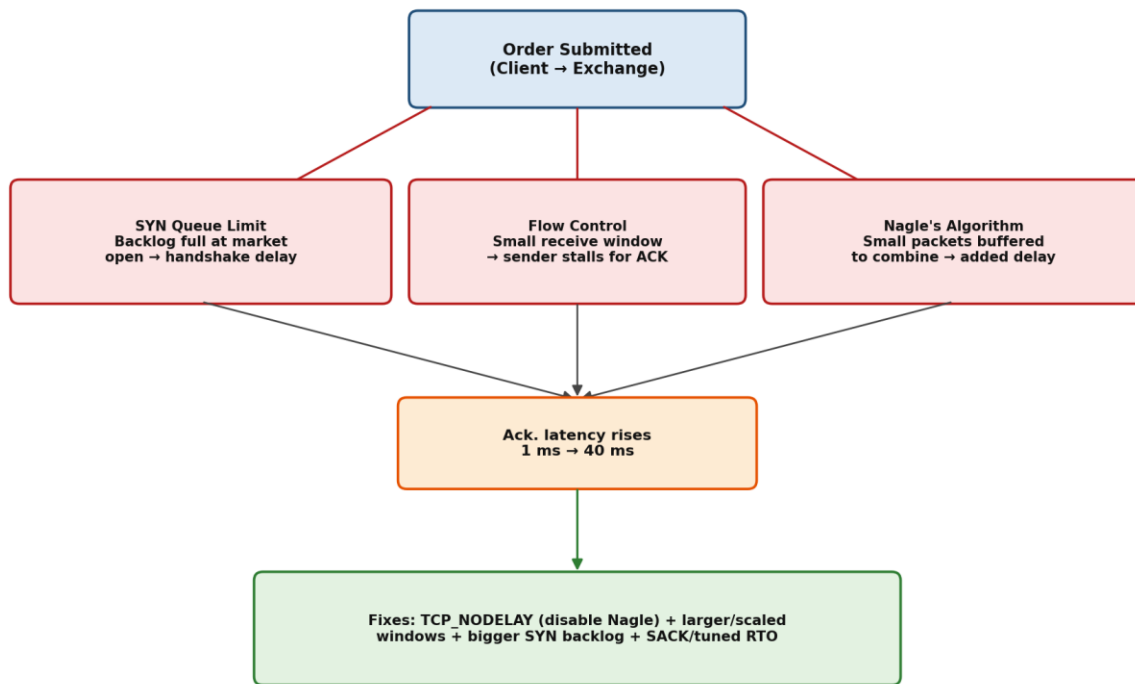


Fig 4.1 — Three TCP factors (SYN queue, flow control, Nagle's algorithm) compounding to raise order acknowledgment latency, and the corresponding fixes

1. TCP-Related Factors Behind the Latency Increase

- Flow control: TCP's receive-window mechanism limits how much unacknowledged data a sender can transmit; if the receiver's window shrinks under heavy load (many concurrent orders), the sender is forced to pause, adding delay to order transmission.
- Nagle's algorithm: This algorithm buffers small outgoing packets to combine them before sending, aiming to reduce overhead. For trading systems that send many small, latency-critical messages, this buffering directly delays acknowledgment and order transmission.
- SYN queue limitations: At peak load, the surge of new connection attempts can exceed the server's SYN backlog queue size, causing connection establishment

delays or retries, which push up the time before the first order/ack can even be exchanged.

2. Effect of Each Factor on High-Frequency Transaction Processing

- Flow control: In high-frequency systems where thousands of small orders are sent per second, a constrained receive window means the sender frequently stalls waiting for ACKs, directly increasing per-order latency from microseconds/1 ms toward tens of milliseconds.
- Nagle's algorithm: Because it delays small packets to batch them (working alongside delayed ACKs), individual order/acknowledgment messages can be held for up to the delayed-ACK timeout, which is highly detrimental when every millisecond affects trade execution and pricing.
- SYN queue limitations: A full SYN backlog causes new client connections to be dropped or retried, so during market-opening bursts, new trading sessions face connection delays exactly when responsiveness matters most, contributing to the rise from 1 ms to 40 ms acknowledgment latency.

3. Recommended TCP Configuration Changes

- Disable Nagle's algorithm by enabling `TCP_NODELAY` on trading sockets so small latency-critical messages are sent immediately instead of being buffered.
- Increase the TCP receive/send buffer (window) sizes and enable TCP window scaling so flow control does not unnecessarily throttle high-frequency small-message traffic.
- Increase the SYN backlog queue size (e.g., tune `net.core.somaxconn` and the `listen()` backlog) and enable SYN cookies to handle connection bursts during market open without dropping requests.

- Consider disabling or tuning delayed ACKs (TCP_QUICKACK) so acknowledgments are not artificially deferred, complementing the Nagle's-algorithm fix.
- Where appropriate, evaluate connection pooling/persistent connections to avoid repeated TCP handshake overhead for high-frequency order submission.
- Monitor and tune retransmission timeout (RTO) settings and enable selective acknowledgment (SACK) to recover faster from any residual packet loss without large latency spikes.